In [1]:
```
import numpy as np
import pandas as pd
from matplotlib import pyplot as plt
import os
from tqdm import tqdm_notebook as tqdm
from operator import itemgetter
from collections import OrderedDict
```

In [2]:
```
# from jupyterthemes import jtplot
# jtplot.style()
```

In [3]:
```
%matplotlib inline
# plt.rcParams['figure.figsize'] = [15, 10]
```

In [4]:
```
data = pd.read_csv('data-edited-mst-date.csv', header =
data.DATE -= 26 # normalise dates to run from 0 to 365
data.MST = data.MST // 60 # convert second timestamp to a
```

In [5]:
```
data.head()
```

	DATE	MST	e0	e1	e2	e3	e4
0	0	436	3.97621	4.44060	4.27966	4.83216	8.42773
1	0	437	4.96652	5.46627	5.29066	5.97006	9.57426
2	0	438	5.75069	6.28833	6.13483	6.76291	10.39120
3	0	439	5.33213	5.86996	5.74418	6.39497	10.08150
4	0	440	5.08743	5.63157	5.47730	6.20357	10.13100

In [6]:
```
# set up constants
ENERGY = data.columns[12]
DATE = 'DATE'
MST = 'MST'
```

In [7]:
```
def get_h_matrix(ENERGY=ENERGY):
    '''
        Returns the energy matrix where h[d][t] is the e
    '''
    h = np.zeros((366, 1440))
    for row in data[[DATE, MST, ENERGY]].values:
        h[int(row[0])][int(row[1])] = row[2]
    return h
```

In [8]:
```
def ewma(h, day=10, alpha=0.5):
    '''
        Returns a list of ewma predictions for given day
    '''
    E = np.zeros(1440)
    for slot in range(1440):
        e = 0.
        for i in range(day-1):
            e = alpha*e + (1-alpha)*h[i][slot]
        E[slot] = e
    return E
```

In [9]:

```
# always beginning from slot K at least
def wcma(h, day=10, alpha=0.5, D=5, K=5):
    '''
        Returns a list of wcma predictions for given day
    '''
    E = np.zeros(1440)
    for slot in range(K, 1440):
        e = 0.
        md = h[0][slot]
        # calculate GAP
        # vector V is a K sized vector
        V = np.zeros(K)
        P = np.zeros(K)
        for k in range(K):
            V[k] = h[day][slot-K+k] / (h.transpose()[slo
            P[k] = float(k)/K
        GAP = np.dot(V, P) / np.sum(P)
        for i in range(day-1):
            # mean over previous D days
            if(i>0): md = h.transpose()[slot][max(0, i-D
            e = alpha*h[i][slot] + (1-alpha)*md*GAP
        E[slot] = e
    return E
```

In [10]:

```
def pro_energy(h, day=10, alpha=0.5):
    '''
        Returns a list of pro-energy predictions for giv
    '''
    def mae(h, d1, d2): return np.abs(h[d1]-h[d2]).mean(
    maes = np.zeros(day)
    for d in range(day):
        maes[d] = mae(h, day-d-1, day)
    # maes[0] is MAE(day-1, d)
    day_idxs = list(enumerate(maes))
    day_idxs.sort(key=itemgetter(1), reverse=True)
    w = np.zeros(day)
    s = np.sum(maes)
    for j in day_idxs:
        w[j[0]] = 1 - (maes[j[0]] / s)

    E = np.zeros(1440)
    for slot in range(1, 1440):
        wp = np.dot(h.transpose()[slot][day-1::-1], w) /
        E[slot] = alpha*h[day][slot-1] + (1-alpha)*wp
    return E
```

In [11]:
```
def mod_pro_energy(h, day=10, alpha=0.5, beta=0.2):
    '''
        Returns a list of pro-energy predictions for giv
    '''
    def mae(h, d1, d2): return np.abs(h[d1]-h[d2]).mean(
    maes = np.zeros(day)
    for d in range(day):
        maes[d] = mae(h, day-d-1, day)
    # maes[0] is MAE(day-1, d)
    day_idxs = list(enumerate(maes))
    day_idxs.sort(key=itemgetter(1), reverse=True)
    w = np.zeros(day)
    s = np.sum(maes)
    for j in day_idxs:
        w[j[0]] = 1 - (maes[j[0]] / s)

    E = np.zeros(1440)
    perr = 0.
    for slot in range(1, 1440):
        perr = beta*perr + (1-beta)*(E[slot-1]-h[day][sl
        wp = np.dot(h.transpose()[slot][day-1::-1], w) /
        E[slot] = alpha*h[day][slot-1] + (1-alpha)*wp -
    return E
```

In [12]:
```
def mae_pred(p1, truth):
    '''
        Helper function to return mean absolute error be
    '''
    return np.abs(p1-truth).mean()
```

In [13]:
```
def get_error(h, p):
    '''
        Helper function to return prediction error ratio
    '''
    p += 1e-5
    return np.abs(1 - (h/p)).mean()
```

In [14]:
```
def plot_error(p, fname=None):
    '''
        Helper function to plot and visualise error rati
    '''
    fig, ax = plt.subplots(1)
    ax.bar(list(p.keys()), list(p.values()))
    ax.set_title("Prediction Error Ratios")
    ax.set_xlabel("Algorithm Used")
    ax.set_ylabel("Error")
    if fname is not None: fig.savefig(fname, bbox_inches
#     ax.set_ylim(0.6, 1.3)
```

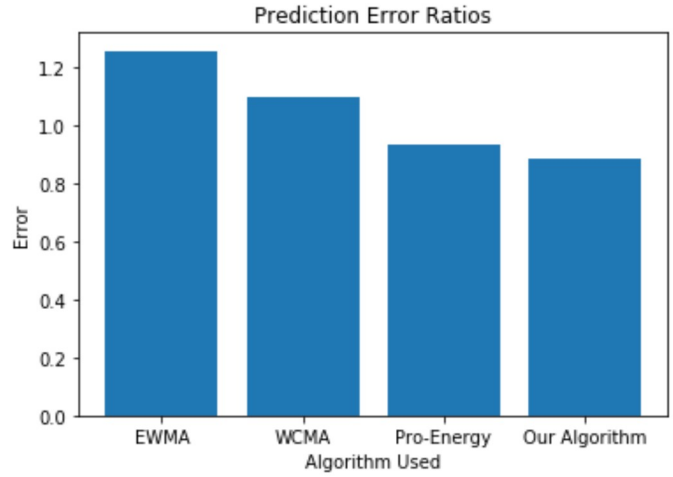
In [15]:
```
def plot_predictions(truth, preds, title, xlim=None, yli
    '''
        Helper function to plot the ground truth against
        Arguments:
            truth: list of truth values
            preds: dictionary containing ("algorithm": p
    '''
    x = [i for i in range(1440)]
    fig, ax = plt.subplots(1, 1)
    ax.plot(x, truth, label="Ground Truth", linewidth=3)
    for p in preds.keys():
        ax.plot(x, preds[p], label=f'{p}', linewidth=1,
    ax.set_title(title)
    ax.set_xlabel("Time Slot [minutes]")
    ax.set_ylabel("Energy [W/m2]")
    ax.legend()
    if xlim is not None: ax.set_xlim(xlim)
    if ylim is not None: ax.set_ylim(ylim)
    if fname is not None: fig.savefig(fname, bbox_inches
```

In [16]:
```
h = get_h_matrix()
```

In [17]:
```
day=10
truth = h[day]
EWMA = ewma(h, day=day)
WCMA = wcma(h, day=day)
PRO = pro_energy(h, day=day)
MOD_PRO = mod_pro_energy(h, day=day)
```

In [18]:
```
errors = OrderedDict({"EWMA": get_error(truth, EWMA),
                      "WCMA": get_error(truth, WCMA),
                      "Pro-Energy": get_error(truth, P
                      "Our Algorithm": get_error(truth
                     })
```

In [19]:
```
plot_error(errors, fname="errors.png")
```



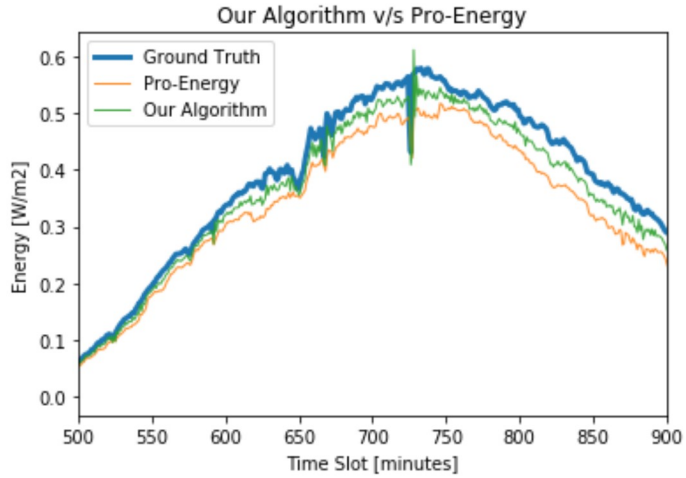
In [20]:
```
errors
```

```
OrderedDict([('EWMA', 1.2545536136392341),
             ('WCMA', 1.093933399644929),
             ('Pro-Energy', 0.9337872464759861),
             ('Our Algorithm', 0.8846498481024798)])
```

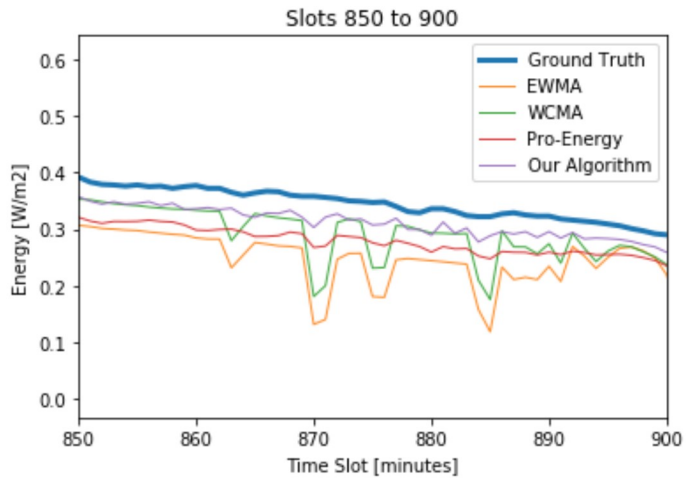
In [25]:
```
(errors['WCMA']-errors['Our Algorithm']) / errors['WCMA'
```

```
19.13128821282711
```

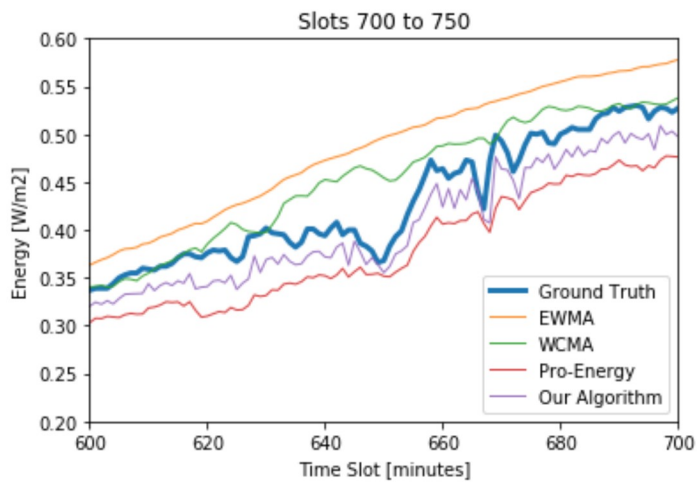
In [42]:
```
plot_predictions(truth, {"Pro-Energy": PRO, "Our Algorit
                 title="Our Algorithm v/s Pro-Energy",
                 xlim=(500, 900), fname="preds_zoomed_ou
```



In [43]:
```
plot_predictions(truth, {"EWMA": EWMA, "WCMA": WCMA, "Pr
                 title="Slots 850 to 900",
                 xlim=(850, 900), fname="preds1.png")
```



In [44]:
```
plot_predictions(truth, {"EWMA": EWMA, "WCMA": WCMA, "Pr
                 title="Slots 700 to 750",
                 xlim=(600, 700),
                 ylim=(0.2, 0.6), fname="preds2.png")
```



In [45]:

```
days = [10, 90, 200, 300]
res_dict= {}
for day in days:
    truth = h[day]
    EWMA = ewma(h, day=day)
    WCMA = wcma(h, day=day)
    PRO = pro_energy(h, day=day)
    MOD_PRO = mod_pro_energy(h, day=day)
    res_dict[f'Day {day}'] = [get_error(truth, EWMA), ge
res_df = pd.DataFrame.from_dict(res_dict, orient='index'
```

In [24]:

```
print("Prediction Error Ratios")
res_df
```

```
Prediction Error Ratios
```

	EWMA	WCMA	PRO	MPE
Day 10	1.254554	1.093933	0.933787	0.884650
Day 90	1.837535	1.787515	2.088974	1.069490
Day 200	1.106148	0.983891	0.975873	0.714068
Day 300	3.020421	1.364042	0.931500	0.917892

In [ ]: